

Deep Learning for NLP – Homework 2

Student name: *Georgios Karteris*
sdi: 2100258

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Abstract	2
2	Data Processing and Analysis	2
2.1	Pre-processing	2
2.2	Analysis	2
2.3	Data Partitioning for Train, Test, and Validation	3
2.4	Vectorization	3
3	Algorithms and Experiments	3
3.1	Experiments	3
3.1.1	Table of Trials	4
3.2	Hyper-parameter Tuning	5
3.3	Optimization techniques	5
3.4	Evaluation	5
3.4.1	ROC curve	5
3.4.2	Learning Curve	6
3.4.3	Confusion matrix	6
4	Results and Overall Analysis	9
4.1	Results Analysis	9
4.2	Comparison with the first project	9
5	Bibliography	9

1. Abstract

The task involves performing sentiment analysis on tweets, aiming to classify them as either positive or negative. This is a binary classification problem that requires a combination of natural language processing (NLP) techniques and machine learning algorithms. To tackle this problem, we employed a systematic approach that included data pre-processing, vectorization using embeddings and classification using Multi-Layer Perceptron models. The process involved multiple stages of experimentation, including baseline model evaluation, vectorization and hyperparameter tuning to optimize the models' performance. The highest score reached was 0.76786.

2. Data Processing and Analysis

2.1. Pre-processing

Preprocessing is a critical step in any NLP pipeline, as it ensures that the data is clean and ready for feature extraction and modeling. For this task, we applied several preprocessing techniques to the raw text data. First, we unescaped HTML characters to handle encoded text properly and removed extra spaces to standardize the formatting. Initially in the first project, we also removed links and special characters (punctuation), which are often irrelevant for sentiment analysis and can introduce noise into the model. However, the utilized glove embeddings utilize this information (mentions, hashtags, punctuation and links) as tagged tokens (e.g., "<user>" instead of "@john"). As such, we compare the two different preprocessing techniques in the experiments. Finally, we converted all text to lowercase to ensure uniformity, as case differences (e.g., "Happy" vs. "happy") do not typically carry semantic meaning in this context. Finally, we tokenize text using the Tweet Tokenizer of the Natural Language Toolkit (NLTK) library.

The two preprocessing techniques can be summarized as follows:

- **Simple preprocessing:** all special characters are removed (including @ and #), along with links. Therefore, mentions and hashtags are treated as regular words.
- **Embedding preprocessing:** special characters are maintained, while links, mentions and hashtags are tagged using < and >.

2.2. Analysis

Most of the analysis is skipped, as the dataset and EDA is identical to the first project. However, we do analyze the number of tokens that could not be mapped to an embedding. We compare the two preprocessing techniques to visualize how much they affect the data. The results are shown in Fig. 1 as a histogram, where the x axis represents the number of tokens that cannot be mapped to an embedding and the y axis represents the corresponding probability across the train, validation and test datasets. For the first preprocessing technique, there are very few tokens that cannot be mapped to an embedding, since most special characters are removed. On the other hand, the second preprocessing technique results in a very wide range of missing tokens. This is because of certain special characters potentially being repeated multiple times in tweets, resulting in many empty embeddings. Therefore, we can conclude that while maintaining

special characters may make use of more embeddings, this may be overshadowed by the increase in unknown tokens. These results are the same across all three dataset partitions.

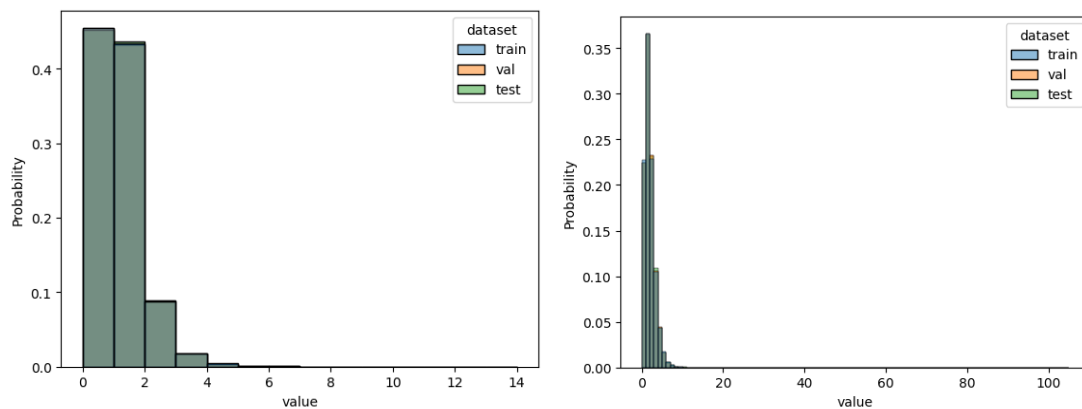


Figure 1: Histogram of the labels and the word count of the tweets in the training dataset.

2.3. Data Partitioning for Train, Test, and Validation

Since the dataset was already partitioned into training, validation, and test sets, we perform training on the training set and evaluation of the validation set. The test set is used only for the final Kaggle submission. Cross-validation is not used because of the lengthy training times of the neural networks.

2.4. Vectorization

Text data needs to be converted into numerical vectors before it can be fed into machine learning models. Word embeddings are dense vector representations of words that capture their word meaning based on context, typically learned from large text collections. Unlike TF-IDF, which represents words without considering context or meaning, word embeddings map similar words to similar vectors, enabling them to capture relationships like similarity or analogy. For this purpose, we map tokens to the Twitter glove embeddings using 200 dimensions.

We integrate these embeddings as the first layer of all developed neural networks, mapping each word to a 200 dimensional vector. In case a token in a tweet is not part of the embedding vocabulary, we replace it with a 0 vector. The mean of each dimension is computed for each tweet, so we can feed the 200 values as input to the neural networks. The embeddings are frozen so that they are not altered during training.

3. Algorithms and Experiments

3.1. Experiments

We structure the first set of experiments as follows:

- **Dummy:** a dummy classifier which predicts classes at random is used as a simple baseline model. All models should perform better than this or they do not have sufficient predictive power to be of practical use.
- **Simple MLP (MLP1):** a Multi-Layer Perceptron (MLP) model consisting of 1 hidden layer with 256 neurons. This model represents an unoptimized baseline, which should hopefully be improved upon by other experiments.
- **Preprocessing (MLP2):** same as MLP1, except it is trained on the dataset with the second preprocessing technique to compare performance.

The first set of experiments provided an indication that the first preprocessing technique resulted in better performance. Therefore, the first preprocessing technique is used for all of the following experiments:

- **Two Layer MLP (MLP3):** a MLP with two layers consisting of 128 and 64 neurons, to assess whether two layers perform better than one.
- **Three Layer MLP (MLP4):** a MLP with three layers consisting of 128, 64 and 32 neurons, to assess whether three layers increase performance.
- **Tuned MLP (MLP5):** a MLP optimized through 20 trials of parameters using the Optuna framework. Each trial selects a random number of layers (1, 3), a random number of neurons for each layer (32, 256), and a random learning rate (10^{-5} , 10^{-1}). All previous models used a learning rate of 10^{-3} . These trials aim to minimize the final validation loss.

All experiments are trained using the Adam optimizer. The models use the ReLu activation function after each hidden layer. The output layer consists of a hidden neuron outputting a numerical value, rather than a probability using the sigmoid activation function. Therefore, the binary cross-entropy loss function with logits is used. All models are trained for 10 epochs and the performance is measured across both training and validation datasets at the end of each epoch. This is done to save time during training and standardize training times. The reported scores are extracted at the end of the 10th epoch on the validation set. The batch size is set to 256.

3.1.1. Table of Trials. Table 1 shows the results for the different trials that took place for training and evaluating the models. The precision, recall and F1-score correspond to the macro averages of the classification report returned by sklearn.

Model	Precision	Recall	F1-Score	Accuracy
Dummy	0.50	0.50	0.50	50.13%
MLP1	0.77	0.77	0.77	76.78%
MLP2	0.76	0.76	0.76	75.61%
MLP3	0.77	0.77	0.77	76.52%
MLP4	0.77	0.77	0.77	76.73%
MLP5	0.76	0.76	0.76	75.88%

Table 1: Trials with the different models and their results on the validation set.

3.2. Hyper-parameter Tuning

Regarding the hyper-parameter tuning performed using Optuna, the resulting model consisted of 3 hidden layers with 108, 102 and 253 neurons, whereas the learning rate was set to approximately $1.8 \cdot 10^{-3}$. During the Optuna study, the model was actually able to reach an accuracy of 77.0%, surpassing all other models. However, when re-training the model with the identified parameters afterwards to observe the learning curves, the model only reached the scores reported in Table 1, perhaps due to different weight initialization or shuffling of the dataset. Regardless, most of the trials did not significantly impact the performance, which was also evident when manually testing different architectures.

3.3. Optimization techniques

The Adam optimizer was used across all experiments with varying learning rates. The loss function utilized was binary cross entropy with logit loss, since the sigmoid activation function was not used for the model's output.

3.4. Evaluation

Overall, the best performing model in terms of accuracy was the simple 1-layer MLP (MLP1) consisting of 256 neurons, although its performance is very comparable to the other MLPs. It also seems that the simple preprocessing technique (removing most of the special characters) ended up slightly outperforming the preprocessing technique that tried to maximize token diversity by tagging special characters. This is not unexpected, due to the apparently different vocabulary between our dataset and the embedding vocabulary, which resulted in many missing tokens when special characters were retained. Thus, we decided to train the rest of the models using the dataset that results from the first preprocessing technique.

The per-class scores did actually differ somewhat between different models. These scores are not displayed in the report, but they are evident in the confusion matrices shown in Fig. 4. For instance, the 2 layer MLP (MLP3) achieved 0.74 recall for the negative class and 0.79 for the positive class, whereas MLP1 achieved 0.80 recall for the negative class and 0.74 for the positive. The mean recall scores and f1-scores end up being similar, but the models have a better performance for different classes in some cases. Overall, considering the simplicity of the model and its performance scores, we consider MLP1 to be the best performing model here.

All in all, the input seems to be the most important factor in model performance, as the different models exhibit more or less equal performance. Perhaps a better preprocessing technique or different embeddings would have further increased scores past the approximately 77% scores shown here. Additionally, in some experiments where embeddings were unfrozen during training, performance did increase past 80% in some cases, but these experiments were not included as the assignment specifically asks for pretrained embeddings.

3.4.1. ROC curve. The ROC curves of all models are depicted in Fig. 2. Most of the models have practically identical performance, except for the MLP trained on the sec-

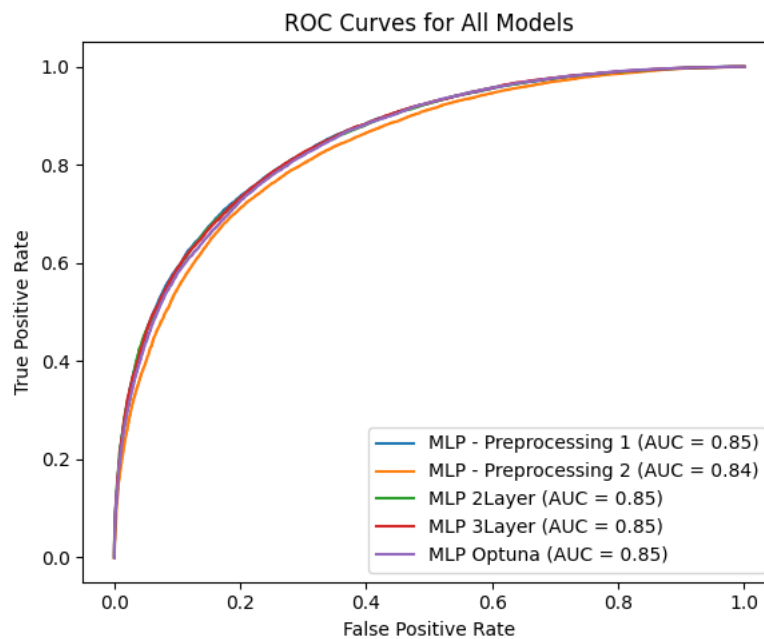


Figure 2: ROC curve of all models.

ond preprocessing technique, which performs slightly worse. The AUC is also identical for most models at 0.85, whereas for MLP2 the AUC is slightly worse at 0.84.

3.4.2. Learning Curve. The learning curve for each model is depicted in Fig. 3. The behavior of each model is rather interesting here:

- MLP1 seems to converge very quickly, but signs of overfitting are not evident here, as loss keeps decreasing and accuracy keeps increasing for both training and validation sets.
- MLP2 scores are a bit more spiky but still no signs of overfitting are present. It can be observed that there is a wider gap between the training and validation scores. Therefore, it may be the case that with further training the performance could have slightly improved. However, in more extensive tests not shown here, the model never exceeded the performance of MLP1 under similar training conditions.
- MLP3 has a somewhat more unstable validation loss than MLP1, but overall performs very similarly to MLP1.
- MLP4 and MLP5 also have very similar performance. However, the gap between the training and validation loss of MLP5 does seem to be slightly bigger. Therefore, perhaps there is slight room for improvement with further training.

3.4.3. Confusion matrix. The confusion matrix extracted from each model for the validation set predictions is depicted in Fig. 4. The confusion matrices do exhibit slight differences between models, as some models tend to favor one class over the other. This is most obvious in the cases of MLP5 (Optuna), where the TN are the highest, vs MLP3 (MLP 2Layer), where the TP are the highest. Nonetheless, all models perform relatively well with similar overall scores.

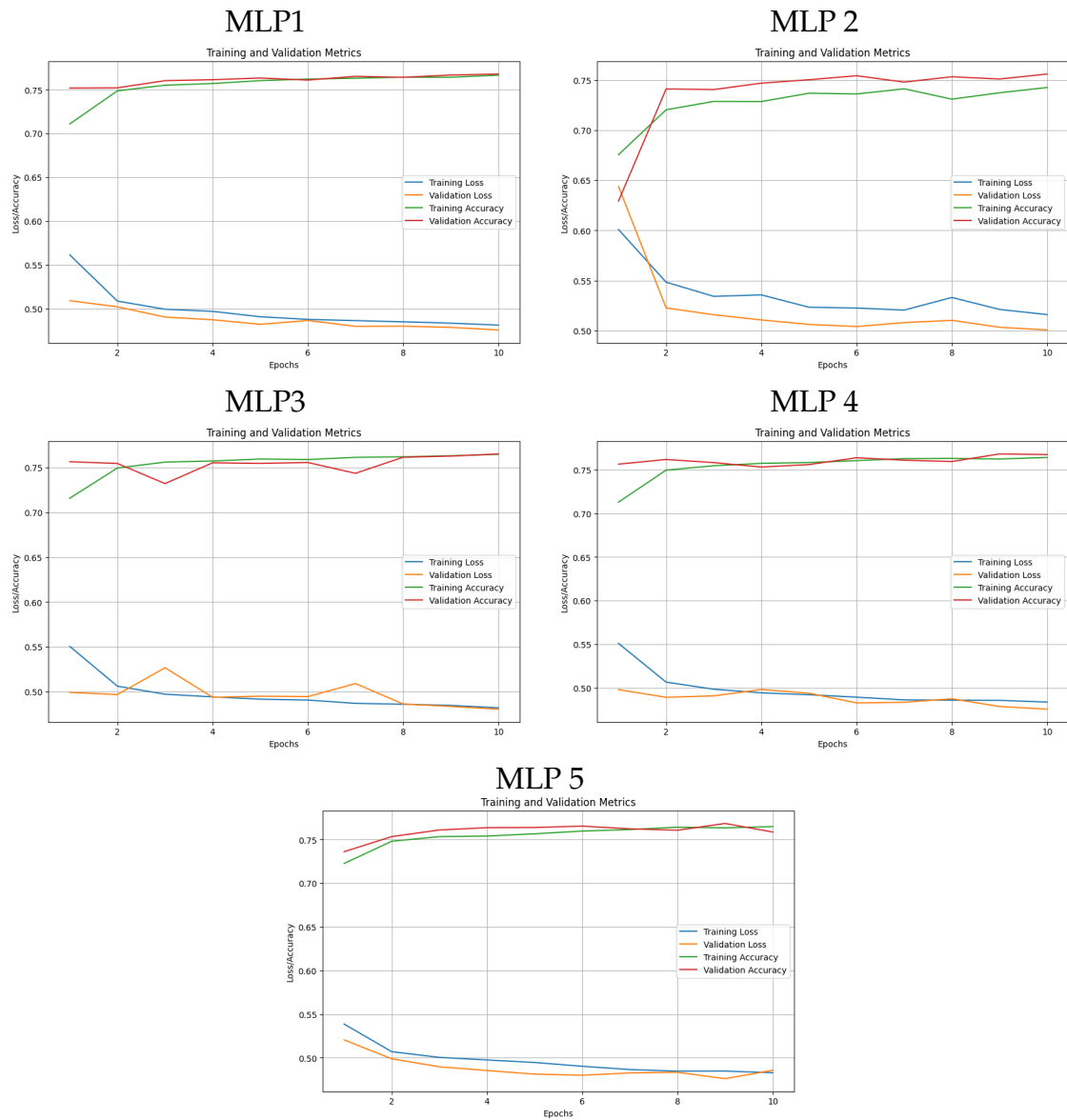


Figure 3: Learning curves for all models.

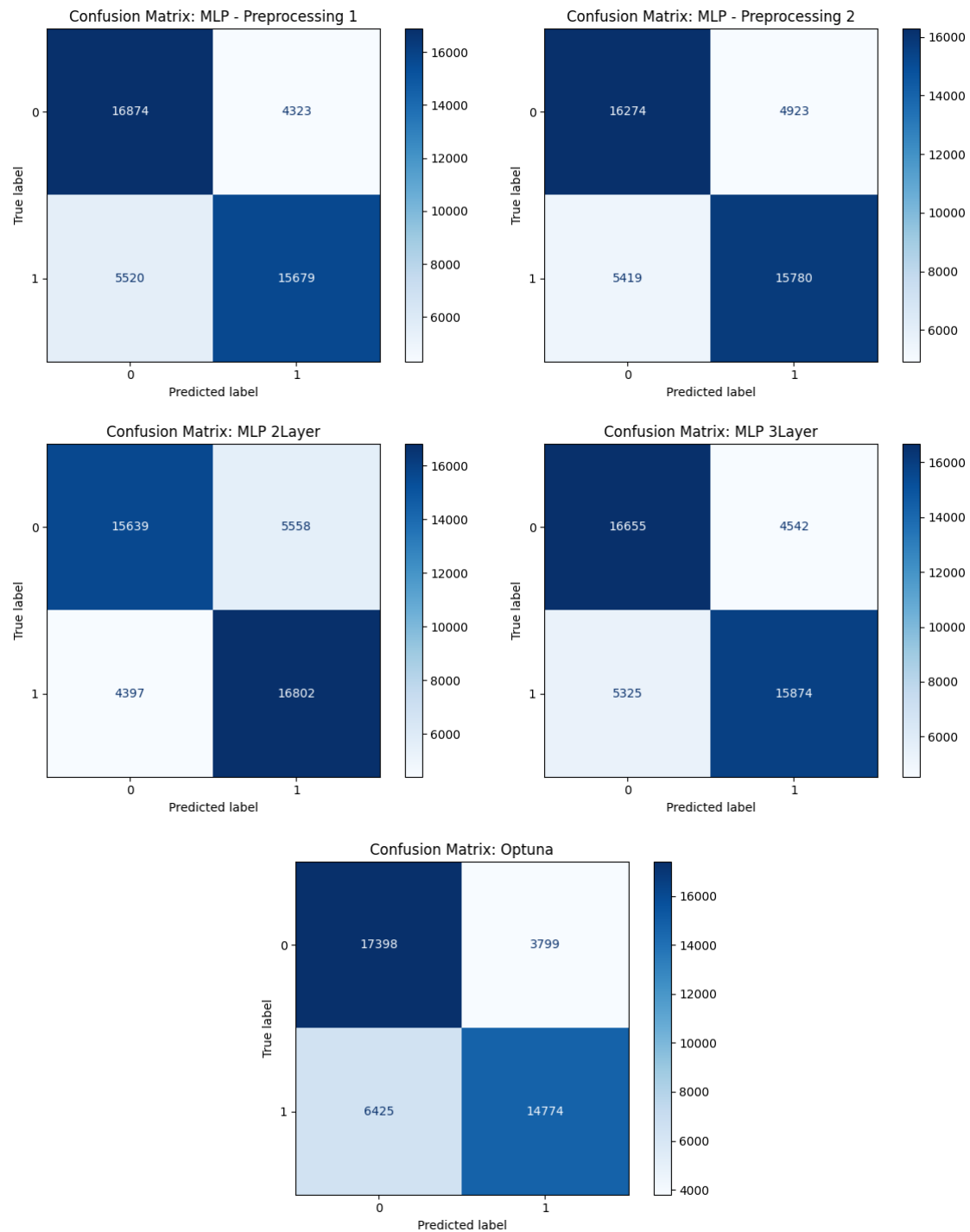


Figure 4: Confusion matrices for all models (except for the dummy).

4. Results and Overall Analysis

4.1. Results Analysis

The results acquired from our approach are overall good, as we achieved an accuracy score of over 76% on the validation set with our best model. This was a significant increase compared to the dummy classifier. Regardless of the model parameters or hyperparameters chosen, the model performance stayed pretty much the same. Therefore, it can be concluded that the text preprocessing process and embeddings chosen affect the performance the most.

4.2. Comparison with the first project

When comparing performance to the first model, it seems that the achieved scores are lower despite using Neural Network-based models, rather than Logistic Regression and embeddings rather than TF-IDF. In this case, it seems that despite the embeddings carrying information about the meaning of words, this did not provide better results than TF-IDF. This is likely the case because many tokens in our dataset could not be mapped to the embedding vocabulary, resulting in many empty (zero) vector representations. Furthermore, we see that optimizing the neural network architecture did not contribute much to overall performance, further highlighting the need for high quality inputs. Due to this, TF-IDF is able to better capture the context of words here for the purposes of classification, as its vocabulary depends on our dataset, rather than a pretrained one. Perhaps training embeddings to better match the needs of our dataset would have been useful in exceeding the performance of TF-IDF models, but this was allowed in the context of this project. TF-IDF is also somewhat superior in that it can be more easily interpreted by humans, when compared to embeddings. This can be beneficial in identifying biases and overfitting.

5. Bibliography

References

- [1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released January 12, 2025.